# Ledger Core: Architecture and Trade-offs

**Design in one line.** Two append-only lists (an event log with outcomes, and a journal of frozen entries) hold all history (plus a write-once event-id index for idempotency). Balances, available balance, authorization state and active fees are derived by folding over them. The ledger has no clock and no I/O.

## 1. Append-only at scale

**What breaks first at 100× volume.** Not memory, but the cost of derivation. Every read is a scan of the whole journal: `balance(account, day)` is O(N). The end-of-day fee check calls it for every account and every day since the start of history, so one close costs roughly accounts × days × N. At today's size that is nothing. At 100× volume and a year of history the nightly close grows with the *square* of history length, and it is the first thing to miss its batch window. `auth_state`, `reverse` and `fee_active` are further linear scans.

**Where state grows without bound.**

- The journal and event log grow forever (correct for audit, but they live in one process list).
- The re-check window grows forever: every close revisits day 1.
- Rejected and declined events are kept, so noisy inputs (retries, declined card attempts) grow the event log faster than the journal.

**Cheapest structural change that defers it.** Two small changes, no new storage engine:

1. **Index and bucket per account.** Keep `journal_by_account` and a running sum per `(account, value_day)`. The balance for day *d* becomes a prefix sum over at most *d* buckets instead of N entries. Index auths by id, entries by event label.
2. **A dirty-day watermark.** Record the earliest `value_day` touched since the last close. The fee re-check starts there instead of day 1. A normal day touches only today, so the close is O(1) per account; a back-valued entry touches just the days from its value date forward, which is exactly the set of balances it can change.

Because history is append-only, a bucket is never edited, only extended, so these indexes are safe caches that can be rebuilt from the journal at any time. The next step after that is partitioning by account (accounts share no state in this design, so it needs no cross-partition coordination), then checkpointing a period-end balance per account so old journal segments can move to cold storage without being read by the nightly run.

## 2. Value-dated entries in a UAE-licensed bank

A value date separates *when the bank learned of something* from *when it takes financial effect*. That makes a booked past period mutable in effect, even though no record is edited. The surface this creates:

- **Customer-facing.** A closed day's balance has already appeared on a statement or in a mobile app. A back-valued entry changes what that day *should have* shown, which can trigger fees or interest the customer did not see at the time. Under the CBUAE consumer-protection framework, charges that arise from the bank's own correction need to be explainable, and often refundable, which is why my model reverses consequential fees by compensating entries rather than keeping them.
- **Interest and profit.** Back-valuing forces recalculation from the value date forward. For Islamic windows or accounts the same mechanics apply to *profit* accrual and need Sharia-board-approved treatment before any recalculation is applied.
- **Regulatory reporting.** Returns and ratios built from closing balances (liquidity, reserve requirements, large-exposure and other CBUAE submissions) are computed as at a date. If a back-valued entry lands

after submission, the bank either restates (with governance, and possibly resubmission) or books the effect in the current period. Both need a defined policy, not a code path someone discovers at quarter end.

- **Finance and audit.** Accounting periods close. A value date inside a closed period must not silently change posted general-ledger balances; the difference has to be booked as a prior-period adjustment or in the current period with an explicit reconciling line.
- **Payments calendar.** UAE business days (Monday–Friday work week, Saturday–Sunday weekend since 2022, public holidays, and Hijri-calendar holiday dates that move each year), UAEFTS and instant-payment cut-offs, and salary-payment (WPS) value dates all determine what a *valid* value date even is.
- **AML and record-keeping.** Screening and transaction-monitoring run on booking time, while reports to the regulator (suspicious activity, cross-border) refer to economic date. Both timestamps must be kept immutable and retrievable for the retention period required by the AML rules (confirm the exact period with compliance).

**One control I would add before go-live: a back-value gate.** Any entry whose value date is earlier than the current open business day is held until (a) the system has computed and shown an *impact preview*: every affected day's recalculated balance, the fees and interest it would create or remove, and any reporting period it touches; (b) a second person (maker-checker) approves it; and (c) a configurable look-back limit has been respected, with anything older than the limit or inside a closed accounting period needing a named controller. This control sits where the damage starts: it turns a silent restatement into a reviewed decision, and its output is the evidence an auditor will ask for.

# 3. Authorization lifecycle

In the model as built, an authorization is `DECLINED` on arrival, `APPROVED` (a hold exists), or `SETTLED` (settlement posted and the whole hold released). A duplicate id is rejected. **An approved authorization has no other exit. It stays active forever.** That is the main gap, and the list below is what production must add, with the behaviour I would mandate.

| End of life | Real-world scenario | Mandated behaviour |
|---|---|---|
| Declined at creation | Not enough available balance, account frozen, limit hit | No hold, outcome recorded with the reason and the balance used. Already built. |
| Expiry | Merchant never presents the item (cancelled order, forgotten pre-auth). Hold lifetimes vary by merchant category and scheme | A timer posts a `HOLD_RELEASE` by system event, with a distinct reason. A late settlement afterwards is treated as unmatched, not silently accepted. |
| Merchant reversal / void | Customer cancels before clearing; full or partial reversal message | Full: release the hold. Partial: reduce the hold to the remaining amount. Both are new entries, never edits. |
| Issuer or customer cancel | Card blocked for fraud, customer disputes the pre-auth | Release or retain by policy, but the decision must be an explicit entry with the actor and reason. |
| Account event | Account closed, frozen by court order, customer deceased | Holds are not auto-released. They convert to a legal or operational block, owned by a named team. |
| Partial or split settlement | Under-settlement (tips, fuel), multiple shipments clearing separately | Define the rule: close on first clearing (as built) or allow multiple clearings until the hold is consumed or expires. Over-settlement needs a tolerance by merchant category. |

| End of life | Real-world scenario | Mandated behaviour |
|---|---|---|
| Unmatched settlement | Clearing with no authorization (offline, stand-in, force-post) | The issuer may already owe the scheme. Route to an exception queue with an owner and an SLA, rather than dropping it, and decide whether it posts to the customer or a suspense account. |
| Invalidated by a back-valued entry | A back-valued debit makes an earlier approval look wrong in hindsight | Do **not** revoke it: the commitment to the scheme was made on information that was true then. Flag it, report it, and let the fee and decline rules react going forward. |

# 4. What I cut, and the risk each defers

| Simplification | Production risk deferred |
|---|---|
| Single-sided ledger (customer sub-ledger, no double entry, no general ledger) | No trial balance or control total; customer balances cannot be proven against the GL. Unbalanced postings go unnoticed. |
| Input validation is minimal (positive finite amounts, days 1–6); event-id check is in-memory only, with no ordering guarantee on input | Redeliveries are ignored only within one process run; after a restart or across channels the same message can post twice. Ids need to be durable and unique per source. |
| No persistence, crash recovery or concurrency control | Lost state on restart. Two writers would race on balance checks (authorize-then-debit). |
| O(N) reads, no indexes (Section 1) | Nightly close misses its window at volume. |
| “Day” is an integer; no business calendar or time zone | Weekends, holidays and cut-off times, and so which value dates are valid, are not modelled. |
| No back-value limit or approval (Section 2) | Any past day can be rewritten without review or period-close protection. |
| Hold expiry, void, multi-clearing and over-settlement tolerance absent (Section 3) | Customer funds stay blocked indefinitely; disputes cannot be represented. |
| Fee only in AED; flat 25.00; no cap, waiver, or agreed overdraft facility | Every negative balance is treated as unarranged. No product tiers or customer-protection caps on charges. |
| One interest rate, one rounding rule, one capitalization, no day-count basis | No tiered rates, no profit-rate rules for Islamic products, no accrual basis (e.g. actual/365) for the regulator or customer disclosures. |
| Window closes at capitalization, with no post-capitalization adjustment path | A genuine late correction after Day 6 is rejected outright; production needs an explicit adjustment workflow with its own approval. |
| Only credits and debits can be reversed; no reversal of fees, settlements or interest as primary events | Complex corrections need manual workarounds outside the model. |
| No identity on entries (who/why) and no maker-checker | No segregation of duties. Weak audit story for a regulated bank. |
| Restated reports are printed, not issued | No customer notification, statement re-issue or regulatory resubmission workflow. |